Testing development and production environment in K8S

Using Kubernetes Namespaces to Manage Environments

One of the advantages that Kubernetes provides is the ability to manage various environments easier and better than traditional deployment strategies. For most nontrivial applications, you have test, staging, and production environments. You can spin up a separate cluster of resources, such as VMs, with the same configuration in staging and production, but that can be costly and managing the differences between the environments can be difficult.

Kubernetes includes a cool feature called [namespaces][4], which enable you to manage different environments within the same cluster. For example, you can have different test and staging environments in the same cluster of machines, potentially saving resources. You can also run different types of server, batch, or other jobs in the same cluster without worrying about them affecting each other.

The Default Namespace

Specifying the namespace is optional in Kubernetes because by default Kubernetes uses the "default" namespace. If you've just created a cluster, you can check that the default namespace exists using this command:

```
$ kubectl get namespaces
NAME          LABELS    STATUS
default                 Active
kube-system             Active
```

Here you can see that the default namespace exists and is active. The status of the namespace is used later when turning down and deleting the namespace.

Creating a New Namespace

You create a namespace in the same way you would any other resource. Create a my-namespace.yaml file and add these contents:

```
kind: Namespace
apiVersion: v1
metadata:
 name: my-namespace
 labels:
   name: my-namespace
```

Then you can run this command to create it:

```
$ kubectl create -f my-namespace.yaml
```

Service Names

With namespaces you can have your apps point to static service endpoints that don't change based on the environment. For instance, your MySQL database service could be named mysql in production and staging even though it runs on the same infrastructure.

This works because each of the resources in the cluster will by default only "see" the other resources in the same namespace. This means that you can avoid naming collisions by creating pods, services, and replication controllers with the same names provided they are in separate namespaces. Within a namespace, short DNS names of services resolve to the IP of the service within that namespace.

You can still access services in other namespaces by looking it up via the full DNS name which takes the form of SERVICE-NAME.NAMESPACE-NAME. So for example, elasticsearch.prod or elasticsearch.canary for the production and canary environments respectively

An Example

Lets look at an example application. Let's say you want to deploy your music store service MyTunes in Kubernetes. You can run the application production and staging environment as well as some one-off apps running in the same cluster. You can get a better idea of what's going on by running some commands:

```
~$ kubectl get namespaces
NAME                    LABELS    STATUS
default                           Active
mytunes-prod                      Active
mytunes-staging                   Active
my-other-app                      Active
```

An Example

Here you can see a few namespaces running. Next let's list the services in staging:

```
~$ kubectl get services --namespace=mytunes-staging
NAME          LABELS                    SELECTOR        IP(S)             PORT(S)
mytunes       name=mytunes,version=1    name=mytunes    10.43.250.14      80/TCP
                                                        104.185.824.125
mysql         name=mysql                name=mysql      10.43.250.63      3306/TCP
```

Next check production:

```
~$ kubectl get services --namespace=mytunes-prod
NAME          LABELS                    SELECTOR        IP(S)             PORT(S)
mytunes       name=mytunes,version=1    name=mytunes    10.43.241.145     80/TCP
                                                        104.199.132.213
mysql         name=mysql                name=mysql      10.43.245.77      3306/TCP
```

An Example

Notice that the IP addresses are different depending on which namespace is used even though the names of the services themselves are the same. This capability makes configuring your app extremely easy—since you only have to point your app at the service name—and has the potential to allow you to configure your app exactly the same in your staging or test environments as you do in production

Caveats

While you can run staging and production environments in the same cluster and save resources and money by doing so, you will need to be careful to set up resource limits so that your staging environment doesn't starve production for CPU, memory, or disk resources. Setting resource limits properly, and testing that they are working takes a lot of time and effort so unless you can measurably save money by running production in the same cluster as staging or test, you may not really want to do that.

Whether or not you run staging and production in the same cluster, namespaces are a great way to partition different apps within the same cluster. Namespaces will also serve as a level where you can apply resource limits so look for more resource management features at the namespace level in the future.

Working with Namespaces

Viewing namespaces

You can list the current namespaces in a cluster using: `kubectl get namespace`

```
NAME              STATUS   AGE
default           Active   1d
kube-node-lease   Active   1d
kube-public       Active   1d
kube-system       Active   1d
```

Working with Namespaces

Viewing namespaces

Kubernetes starts with four initial namespaces:

- `default` The default namespace for objects with no other namespace
- `kube-system` The namespace for objects created by the Kubernetes system
- `kube-public` This namespace is created automatically and is readable by all users (including those not authenticated). This namespace is mostly reserved for cluster usage, in case that some resources should be visible and readable publicly throughout the whole cluster. The public aspect of this namespace is only a convention, not a requirement.
- `kube-node-lease` This namespace for the lease objects associated with each node which improves the performance of the node heartbeats as the cluster scales.

Working with Namespaces

Setting the namespace for a request

To set the namespace for a current request, use the `--namespace` flag.

For example:

```
kubectl run nginx --image=nginx --namespace=<insert-namespace-name-here>
kubectl get pods --namespace=<insert-namespace-name-here>
```

Working with Namespaces

Setting the namespace preference

You can permanently save the namespace for all subsequent kubectl commands in that context.

```
kubectl config set-context --current --namespace=<insert-namespace-name-here>
# Validate it
kubectl config view --minify | grep namespace:
```

Namespaces and DNS

When you create a Service, it creates a corresponding DNS entry. This entry is of the form `<service-name>.<namespace-name>.svc.cluster.local`, which means that if a container just uses `<service-name>`, it will resolve to the service which is local to a namespace. This is useful for using the same configuration across multiple namespaces such as Development, Staging and Production. If you want to reach across namespaces, you need to use the fully qualified domain name (FQDN).

Not All Objects are in a Namespace

Most Kubernetes resources (e.g. pods, services, replication controllers, and others) are in some namespaces. However namespace resources are not themselves in a namespace. And low-level resources, such as nodes and persistentVolumes, are not in any namespace.

To see which Kubernetes resources are and aren't in a namespace:

```
# In a namespace
kubectl api-resources --namespaced=true

# Not in a namespace
kubectl api-resources --namespaced=false
```

Q&A